1 / THE BIG IDEA

When will the bus arrive?

Your goal: use past RTS bus records to predict how long a bus will take to reach its next stop.

You will work with real data, clean it, build a simple prediction, and test whether machine learning makes that prediction better.

One example explains the project

A bus reaches Stop A at **6:05:27**. It reaches Stop B at **6:06:53**. The time between them is **86 seconds**.

Travel time = arrival at B − arrival at A

Predicted arrival at B
= arrival at A + predicted travel time

If your model predicts **90 seconds**, the predicted arrival at B is **6:06:57**. That is only **4 seconds** later than the actual arrival. The prediction is an illustration, not a result from a fitted model.

The question you are trying to answer

Can a model predict the next stop's travel time better than simply using the usual travel time for that pair of stops?

You are predicting the time between arrivals. This may include waiting and boarding at the first stop, as well as driving.

Your path through the project

1. Understand the records you have.
2. Turn reliable stop records into travel-time examples.
3. Build a simple comparison, then try a model.
4. Test on later dates and explain what you learned.

Start small: use Route 7 first. You do not need a live bus API, a website, or a complicated model to complete this project.

2 / MEET YOUR DATA

What does one row mean?

One raw row is a recorded stop visit by a bus. It is not an entire bus journey. A bus can visit the same stop many times in one day.

The files available to your group

Route	Raw rows	Dates observed	Date range in 2023
7: start here	14,685	17	March 1–23
3: add later	8,642	17	March 1–23
11: add later	23,365	17	March 1–23
711: optional	4,076	7	March 4–19

These are incomplete samples of March 2023, not full months or a year of data. The number of usable examples will be smaller after cleaning.

A few real Route 7 records

These selected columns describe bus **2105** on **March 1, 2023**.

Stop ID	Stop order	Arrival	On	Off
1	0	06:05:27	7	0
292	1	06:06:53	0	0
293	2	06:07:12	0	0

On / Off means passengers boarding / leaving the bus.

Focus on these columns first

- **Route, Bus, Date, Arrive:** which bus, which day, and what time?
- **Stop ID, Stop Name, Stop Seq ID:** which stop and where in the sequence?

Other columns describe passenger load, passenger miles, interstop distance, latitude, and longitude. Explore these after the basic pipeline works.

Try this: load `RTS_Route7_March2023.xlsx` with pandas. Look at `df.head()`, `df.info()`, and `df.isna().sum()`. Count dates, buses, and stops.

What to turn in
One short notebook showing the first rows, column types, missing values, and three observations or questions about the data.

3 / MAKE GOOD EXAMPLES

Clean first. Then calculate travel time.

Your most important job: make sure two rows really describe consecutive stops on the same journey.

Work through these steps

1. **Keep the original file unchanged.** Save cleaned data separately.
2. **Check the values.** Parse dates and times. Check missing entries, repeated records, negative passenger counts, and unusual stop sequences. Do not automatically replace missing values with zero.
3. **Build arrival timestamps.** Combine Date and Arrive. Inspect midnight crossings instead of automatically adding a day to strange intervals.
4. **Identify separate journeys.** Within a route, bus, and date, inspect times and stop order. A restart in stop order, a terminal visit, or a large time gap may mark a new journey. Check examples with your group leader. The file does not supply a trip ID for you.
5. **Pair consecutive stops within a verified journey.** Subtract arrival at A from arrival at B. Do not pair the end of one journey with the start of the next. Do not shift the whole dataframe by one row.

The new table you are building

Route	From stop	To stop	Travel time (seconds)
7	1	292	86
7	292	293	19

These examples come from page 2; confirm journey continuity before accepting them. Also retain date, bus, journey ID, and both timestamps for checking.

Pause and discuss: What could a zero, negative, or very long travel time mean? Is it a duplicate, a missed stop, a new journey, or a real delay? Record why you remove a row; do not delete every large value.

What to turn in
A cleaned stop-pair table, five manually checked examples, and a short log of what you removed and why. Review this with your leader before modeling.

4 / CHOOSE YOUR INPUTS

What can help the model predict?

A **feature** is information the model uses to make a prediction. The **target** is what it is trying to predict: travel time in seconds.

Start with four simple features

Feature	Why it might help
Route	Different routes may behave differently.
From-stop + to-stop	Some segments take longer than others. Direction matters.
Hour of day	Traffic and bus activity can change during the day.
Day of week	Travel patterns may differ across weekdays and weekends.

Treat route and stop IDs as labels, not quantities. Stop 292 is not "292 times" anything. In scikit-learn, an encoder converts these labels into a form a model can use.

Then try one addition at a time

- **Previous segment time:** was the bus already moving slowly before it reached A?
- **Segment distance:** do longer segments take longer? Verify the distance units and which segment the field describes first.
- **Weather:** does the most recent available rain report help? This is optional and needs timestamp matching.

The rule: no peeking into the future

Imagine making the prediction **the instant the bus arrives at A**. Use only information you could know then.

Do not use arrival at B or future-stop passenger counts as inputs. Passenger counts at A may only be known after boarding finishes, so leave them out of the first model. Determine the expected next stop from a verified route pattern, rather than letting a future observation reveal it.

Try this: compare the basic model with the same model plus previous segment time. Did the extra feature improve predictions on later dates?

What to turn in
A small feature table: feature name, how you created it, and why it would be available when the bus arrives at A.

5 / BUILD AND TEST

Does machine learning actually help?

1. Make a simple prediction first

For each directed stop pair, calculate its **median travel time using training dates only**. The median is the middle value when times are sorted. Predict that same value whenever that pair appears.

This is your **baseline**: the simple method your model must beat. For a pair never seen in training, use the route's training median instead.

2. Try one model, then a second if time allows

Start with **ridge regression**, a simple regression model. Then try a **random forest**, which combines many decision trees. Use scikit-learn preprocessing to handle category labels and missing values. Keep model settings modest; ask your leader for help with the first pipeline.

3. Test on later dates

For Route 7's 17 observed dates, put them in order:

- **First 10 dates: training.** Learn medians, preprocessing, and models.
- **Next 3 dates: validation.** Choose features and model settings.
- **Last 4 dates: test.** Evaluate your final choice once.

Do not randomly mix rows from the same journey across these groups. Do not calculate medians or fill missing values using the full dataset. Your test dates must stay out of model and feature decisions.

4. Measure mistakes in seconds

Use **mean absolute error (MAE)**: calculate the absolute error of each prediction, then average those errors. Smaller is better.

Example: errors of 10, 20, and 30 seconds give an MAE of **20 seconds**. Compare baseline and models on the same test examples. Also show which stop pairs or times of day have the largest errors and how many examples they contain.

Questions for your report: Did the model beat the baseline? Where did it struggle? What might explain that? Would you trust the result outside these March 2023 dates?

What to turn in
A baseline-versus-model results table, two error plots, and a short explanation. It is okay if the baseline wins: explain what you learned.

6 / YOUR WEEKLY CHECKLIST

A practical six-week plan

Week	Focus	Bring to your check-in
1	Set up Python. Explore Route 7.	Data overview and questions.
2	Clean records. Verify journeys and stop pairs.	Cleaned table and five checked pairs.
3	Set aside later dates. Build the median baseline.	Split dates and baseline errors.
4	Fit a model. Try one extra feature.	Validation comparison.
5	Investigate mistakes. Add Routes 3 and 11 only if ready.	Error plots and explanations.
6	Evaluate the final model. Prepare your report and demonstration.	Reproducible work and final presentation.

What a finished project looks like

- Notebooks someone else can run from the original data.
- A short record of cleaning choices and feature definitions.
- A fair baseline/model comparison on held-out dates.
- One example showing predicted arrival, actual arrival, and error.
- An honest explanation of what the data can and cannot tell us.

If you have extra time: explore the downloaded historical schedule (GTFS) or March 2023 airport weather. Schedule matching is a separate challenge. Monthly ridership and TBEST passenger summaries describe demand; they do not provide extra individual arrival times. Keep these extensions optional.

If you get stuck: bring a small example, the exact problem, and what you tried to your group leader. Unreliable journey labels are a reason to narrow the study to verified stop pairs before adding complexity.

Data credit and sources. Ride-check workbooks: Richard Borzi, University of Florida; data supplied by Gainesville RTS; CC BY 4.0. Route 7, Route 3, Route 11, Route 711. Inventory checked October 18, 2026. This is a historical study; no model results are claimed in this handout.